

TP Développement d'APIs Flask Simulation d'Organes Humains

Génération de Données Médicales Synthétiques

Table des matières

1	Présentation du Projet	3
1.1	Objectif Pédagogique	3
1.2	Compétences Visées	3
2	Structure du Projet	3
2.1	Architecture Générale	3
2.2	Répartition des Organes	4
2.2.1	Groupe 1 - Systèmes Vitaux	4
2.2.2	Groupe 2 - Systèmes de Support	4
2.2.3	Groupe 3 - Systèmes Spécialisés	4
3	Cahier des Charges	4
3.1	Exigences Techniques Communes	4
3.1.1	Structure de Base Obligatoire	4
3.1.2	Endpoints Obligatoires	4
3.2	Spécifications par Type d'API	5
3.2.1	API Cardiaque (Exemple Détaillé)	5
3.2.2	API Respiratoire	5
3.2.3	API Neurologique	5
4	Consignes de Développement	6
4.1	Installation et Configuration	6
4.1.1	Environnement de Développement	6
4.1.2	Structure du Code	6
4.2	Génération de Données Synthétiques	6
4.2.1	Principes à Respecter	6
4.2.2	Facteurs d'Influence	6
5	Documentation et Tests	6
5.1	Documentation Requise	6
5.1.1	Documentation Technique	6
5.1.2	Documentation des APIs	7
5.2	Validation et Tests	7
5.2.1	Tests à Implémenter	7
5.2.2	Validation des Données	7
6	API d'Orchestration	7
6.1	Objectif	7
6.2	Fonctionnalités Attendues	7
7	Évaluation et Livrables	8
7.1	Livrables Attendus	8
7.1.1	Pour Chaque API	8
7.1.2	Livrable Final	8

8	Recommandations et Bonnes Pratiques	8
8.1	Conseils de Développement	8
8.2	Aspects Éthiques	8
9	Ressources et Références	8
9.1	Ressources Techniques	8
9.2	Références Médicales	9
9.3	Outils Recommandés	9

1 Présentation du Projet

1.1 Objectif Pédagogique

Ce TP a pour objectif de développer un système distribué d'APIs Flask simulant le fonctionnement des organes humains et générant des données synthétiques médicales réalistes.

1.2 Compétences Visées

- Développement d'APIs REST avec Flask
- Génération de données synthétiques réalistes
- Architecture microservices
- Documentation d'API
- Tests unitaires
- Travail en équipe

2 Structure du Projet

2.1 Architecture Générale

Chaque équipe développera plusieurs APIs représentant différents organes humains. La structure recommandée est la suivante :

- . projet-organes/.
- .- cardiac-api/.
- .— app.py.
- .— requirements.txt.
- .— README.md.
- .- respiratory-api/.
- .— app.py.
- .— requirements.txt.
- .— README.md.
- .- neural-api/.
- .— app.py.
- .— requirements.txt.
- .— README.md.
- .- orchestration-api/.
- .— app.py.
- .— requirements.txt.
- .— README.md.
- .- documentation/.
- .— API_DOCUMENTATION.md.

2.2 Répartition des Organes

2.2.1 Groupe 1 - Systèmes Vitaux

- **Cœur** (Cardiac API) - Système cardiovasculaire
- **Poumons** (Respiratory API) - Système respiratoire
- **Cerveau** (Neural API) - Système nerveux central
- **Foie** (Hepatic API) - Système métabolique

2.2.2 Groupe 2 - Systèmes de Support

- **Reins** (Renal API) - Système urinaire
- **Système Digestif** (Digestive API) - Appareil digestif
- **Peau** (Dermal API) - Système tégumentaire
- **Système Endocrinien** (Endocrine API) - Système hormonal

2.2.3 Groupe 3 - Systèmes Spécialisés

- **Système Immunitaire** (Immune API) - Défenses immunitaires
- **Système Musculosquelettique** (Musculoskeletal API) - Muscles et squelette
- **Système Sanguin** (Hematological API) - Composition du sang
- **Appareil Reproducteur** (Reproductive API) - Système reproductif

3 Cahier des Charges

3.1 Exigences Techniques Communes

3.1.1 Structure de Base Obligatoire

Chaque API doit implémenter la structure suivante :

- Classe de simulation dédiée à l'organe
- Génération de données synthétiques réalistes
- Gestion des paramètres physiologiques
- Simulation de conditions pathologiques

3.1.2 Endpoints Obligatoires

Pour chaque organe, les endpoints suivants doivent être implémentés :

- GET /api/[organ]/status - Statut de l'organe
- GET /api/[organ]/data - Données synthétiques courantes
- POST /api/[organ]/simulate/<condition> - Simulation de pathologie
- GET /api/[organ]/parameters - Paramètres de simulation disponibles

3.2 Spécifications par Type d'API

3.2.1 API Cardiaque (Exemple Détaillé)

- **Données à générer :**
 - Fréquence cardiaque (battements/minute)
 - Pression artérielle (systolique/diastolique)
 - Rythme ECG (normal, tachycardie, bradycardie)
 - Saturation en oxygène
 - Débit cardiaque
- **Conditions à simuler :**
 - Tachycardie
 - Bradycardie
 - Arythmie
 - Hypertension

3.2.2 API Respiratoire

- **Données à générer :**
 - Fréquence respiratoire
 - Volume courant
 - Capacité vitale
 - Gaz du sang (PaO₂, PaCO₂, pH)
 - Saturation en oxygène
- **Conditions à simuler :**
 - Asthme
 - BPCO
 - Hyperventilation
 - Apnée

3.2.3 API Neurologique

- **Données à générer :**
 - Rythme EEG (alpha, bêta, thêta, delta)
 - Niveau d'activité cérébrale
 - Niveaux de neurotransmetteurs
 - Temps de réaction
 - Activité corticale
- **Conditions à simuler :**
 - Épilepsie
 - Migraine
 - Trouble du sommeil
 - Stress

4 Consignes de Développement

4.1 Installation et Configuration

4.1.1 Environnement de Développement

1. Créer un environnement virtuel Python
2. Installer les dépendances nécessaires
3. Configurer le fichier requirements.txt
4. Choisir des ports distincts pour chaque API

4.1.2 Structure du Code

- Séparation claire entre logique métier et routes API
- Utilisation de classes pour la simulation
- Gestion propre des erreurs
- Code commenté et documenté

4.2 Génération de Données Synthétiques

4.2.1 Principes à Respecter

- **Réalisme** : Baser les données sur des valeurs physiologiques réelles
- **Variabilité** : Introduire une variabilité naturelle dans les données
- **Paramétrage** : Permettre la modification des paramètres de simulation
- **Cohérence** : Assurer la cohérence entre les différentes mesures

4.2.2 Facteurs d’Influence

Les données générées doivent prendre en compte :

- Âge du patient simulé
- Sexe
- État de santé général
- Conditions pathologiques activées
- Activité physique simulée

5 Documentation et Tests

5.1 Documentation Requise

5.1.1 Documentation Technique

Chaque API doit inclure :

- Fichier README.md avec :
 - Description de l’organe simulé des endpoints disponibles

- Exemples d'utilisation
- Paramètres acceptés
- Documentation des modèles de données
- Guide d'installation et démarrage

5.1.2 Documentation des APIs

- Description détaillée de chaque endpoint
- Exemples de requêtes et réponses
- Codes d'erreur et leur signification
- Paramètres de requête disponibles

5.2 Validation et Tests

5.2.1 Tests à Implémenter

- Tests unitaires pour la génération de données
- Tests d'intégration des endpoints
- Validation des paramètres d'entrée
- Tests de performance basiques

5.2.2 Validation des Données

- Vérification des plages physiologiques
- Cohérence interne des données générées
- Gestion des cas limites

6 API d'Orchestration

6.1 Objectif

Développer une API centralisée permettant :

- L'agrégation des données de tous les organes
- La surveillance de l'état général du "système"
- La coordination des simulations
- Une vue d'ensemble des paramètres physiologiques

6.2 Fonctionnalités Attendues

- Endpoint d'état général du système
- Récupération consolidée des données
- Simulation synchronisée de conditions
- Surveillance de la santé des APIs

7 Évaluation et Livrables

7.1 Livrables Attendus

7.1.1 Pour Chaque API

- Code source complet et commenté
- Documentation technique
- Fichier requirements.txt
- Exemples d'utilisation

7.1.2 Livrable Final

- Dépôt Git contenant l'ensemble du projet
- Documentation complète
- Présentation du travail
- Démonstration fonctionnelle

8 Recommandations et Bonnes Pratiques

8.1 Conseils de Développement

- Commencer par un organe simple pour valider l'approche
- Tester régulièrement chaque endpoint
- Utiliser Git pour le versionnement
- Documenter au fur et à mesure du développement

8.2 Aspects Éthiques

- Mentionner clairement le caractère synthétique des données
- Éviter toute confusion avec des données médicales réelles
- Respecter les sensibilités liées aux données de santé
- Usage strictement pédagogique

9 Ressources et Références

9.1 Ressources Techniques

- Documentation Flask : <https://flask.palletsprojects.com/>
- Standards REST API
- Principes de physiologie humaine

9.2 Références Médicales

- Valeurs physiologiques normales
- Symptomatologie des pathologies
- Paramètres d'observation clinique

9.3 Outils Recommandés

- Postman pour tester les APIs
- Git pour le versionnement
- VS Code avec extensions Python
- Plateforme de déploiement (Heroku, Render, etc.)

Remarque importante : Ce projet a un objectif purement pédagogique. Les données générées sont synthétiques et ne doivent en aucun cas être utilisées pour des diagnostics médicaux réels.